MIVA
OPEN UNIVERSITY

COS 201

Computer Programming I



© Copyright 2026 MIVA Open University All Rights Reserved

MIVA OPEN UNIVERSITY

HEADQUARTERS

PLOT 1059, O.P. FINGESI ROAD,

UTAKO, ABUJA

© Copyright 2026 MIVA Open University All Rights Reserved

Course Development Team

Course Developer/Writer: Dr Emeka Ogbuju, PhD Artificial Intelligence

Language Editor: Udochi Obiukwu, MA English Language

Instructional Designer: Dr Morolake Lawrence, PhD Computer Science

© Copyright 2026 MIVA Open University All Rights Reserved

Table of Contents

Table of Contents 5
List of Figures 7
List of Tables 8
Course Introduction 9
Course Aims 9
Course Learning Outcomes 10
Study Session One 13
Introduction 14
Learning Outcomes for Study Session 1 15
1.1. Types of Programming Language 16
1.1.1. Machine Language 16
1.1.2. Low-Level Language (Assembly Language) 16
1.1.3. High-Level Language 16
1.2. What is Python? 17
1.3. Python vs Other Languages 18
1.3.1. Python vs. Java 18
1.3.2. Python Vs JavaScript 18
1.3.3. Python Vs Perl 19
1.3.4. Python Vs C++ 19
Figure 1.1 Stack overflow developer survey 2018 20
1.4. Installation of Python 21
Figure 1.2: Python installation interface 22
Conclusion 24
Summary 24
Study Session Two 29
Introduction 30
Learning Outcomes for Study Session 2 30
2.1. Input and Output Statements 32
2.1.1. Input statement syntax and examples 32
2.1.2. Output statement syntax and examples 32
2.2. Comments, Data types and Operators 33
2.2.1. Comments 33
Figure 2.1 Single-line comments 34
Figure 2.2 Approach 1—Starting each line with a # 35

© Copyright 2026 MIVA Open University All Rights Reserved

Figure 2.3 Approach 2—Using string Literals 36
Figure 2.4: Python Docstrings 37
2.2.2. Data Type 38
Table 2.1: Data Types and their Descriptions 38
2.2.3. Operators 41
Table 2.2: Assigned Operators 42
Table 2.3: Arithmetic Operators 43
Table 2.4: Logical Operators 43
Table 2.5: Comparison Operators 44
Table 2.6: Identity Operators 44
Table 2.7: Membership Operators 45
Table 2.8: Bitwise Operators 45
2.3 General Examples 46
Conclusion 49
Summary 49
Study Session Three **54**
Introduction 55
Learning Outcomes for Study Session 3 56
3.1. Decision Statement 57
3.1.1. IF – Else Statement 57
Figure 3.1 IF - Else Statement 57
Figure 3.2 IF Syntax 58
3.1.2. Nested IF Statement 59
3.2. Loops, Range, Break And Continue 61
3.2.1. FOR Loop 62
Figure 3.3: FOR Loop 62
3.2.2. WHILE Loop 63
Figure 3.4 While loop 64
3.2.3 Range, Break and Continue 65
Table 3.1: Range Parameters And Descriptions 65
Figure 3.5: range() function code example 66
Conclusion 67
Summary 67
Study Session Four **72**
Introduction 73

© Copyright 2026 MIVA Open University All Rights Reserved

Learning Outcomes for Study Session 4 74
4.1. Functions 75
4.1.1. Declaration of Function 75
4.1.2. Scope of Function 76
Figure 4.1: Example Code of Local Scope 77
Figure 4.2: Example Code of Global Scope 78
4.2. Arrays 79
4.2.1. Declaration of Arrays 80
Figure 4.3: Array 80
4.2.2. Types of Arrays 81
Figure 4.2 One-directional array 81
Figure 4.3 Two-directional array 82
Table 4.1: Array Inbuilt Methods For Manipulating A List 83
Conclusion 84
Summary 84
Study Session Five **89**
Introduction 90
Learning Outcomes for Study Session 5 91
5.1. How To Read, Write And Append Files 92
5.1.1. Opening a File 92
5.1.2. Writing a File 93
5.1.3. Reading a File 93
Table 5.1: File Object Processes and Functions 94
5.2. Exception Handling 96
5.2.1. Catching Exceptions 96
5.2.2. Except Clause with No Exceptions 98
5.2.3. Multiple Exceptions handling 98
Figure 5.1: Multiple exceptions handling code example 99
5.2.4. User-Defined Exceptions 99
Conclusion 99
Summary 100
References/Further Readings 105
Glossary 107

© Copyright 2026 MIVA Open University All Rights Reserved

List of Figures

Figure 1.1 Stack overflow developer survey 2018 20
Figure 1.2: Python installation interface 22
Figure 2.1 Single-line comments 34
Figure 2.2 Approach 1—Starting each line with a # 35
Figure 2.3 Approach 2—Using string Literals 36
Figure 2.4: Python Docstrings 37
Figure 3.1 IF - Else Statement 57
Figure 3.2 IF Syntax 58
Figure 3.3: FOR Loop 62
Figure 3.4 While loop 64
Figure 3.5: range() function code example 66
Figure 4.1: Example Code of Local Scope 77
Figure 4.2: Example Code of Global Scope 78
Figure 4.3: Array 80
Figure 4.2 One-directional array 81
Figure 4.3 Two-directional array 82
Figure 5.1: Multiple exceptions handling code example 99

List of Tables

Table 2.1: Data Types and their Descriptions 38
Table 2.2: Assigned Operators 42
Table 2.3: Arithmetic Operators 43
Table 2.4: Logical Operators 43
Table 2.5: Comparison Operators 44
Table 2.6: Identity Operators 44
Table 2.7: Membership Operators 45
Table 2.8: Bitwise Operators 45
Table 3.1: Range Parameters And Descriptions 65
Table 4.1: Array Inbuilt Methods For Manipulating A List 83
Table 5.1: File Object Processes and Functions 94

© Copyright 2026 MIVA Open University All Rights Reserved

Course Introduction

Computer Programming I is a 3-credit unit course designed for students pursuing degrees in Computer Science, BSc Information Technology, BSc Software Engineering, and related disciplines. It consists of five sessions and 12 study units, focusing on problem-solving strategies, algorithm development, program design, coding, debugging, and documentation. Scripting languages such as Python or QBasic may be utilised for teaching.

The course covers fundamental programming constructs, syntax, semantics, variables, data types, expressions, assignments, I/O operations, conditional and interactive control structures, functions, parameter passing, and file handling. It introduces students to automated information processing, laying the conceptual foundation necessary for programme comprehension and creation.

Course Aims

This course aims to:

1. Define programming and identify the primary categories of programming languages.
2. Differentiate between high-level language, low-level language, and machine language.
3. Explain Python and articulate its advantages over other programming languages.
4. Independently install Python on their computers.
5. Write Python code for input and output and execute it within their installed Python environment.

© Copyright 2026 MIVA Open University All Rights Reserved

6. Skilfully compose single-line or multiple-line comments and demonstrate proficiency in all required data types and operators for Python programming.
7. Utilise decision statements such as the IF-ELSE and nested IF statements in program writing.
8. Implement codes for program segment repetition using the FOR-loop statement and the WHILE loop statement.
9. Create and integrate functions in Python programs.
10. Grasp the concept of arrays, understand their types, and proficiently write codes employing one-dimensional and two-dimensional arrays.

Course Learning Outcomes

By the end of this course, you should be able to:

1. Explain the fundamental concepts of programming, including the distinction between machine, low-level, and high-level languages.
2. Install and configure the Python programming environment to write and execute simple programs.
3. Demonstrate proficiency in Python syntax by using input/output operations, comments, data types, and operators effectively.
4. Apply decision-making and control structures, such as if-else, for, and while loops, to solve computational problems.
5. Design and implement Python functions, differentiating between built-in and user-defined functions, and explain the concept of variable scope.
6. Develop and manipulate one-dimensional and two-dimensional arrays to store and process data efficiently.

© Copyright 2026 MIVA Open University All Rights Reserved

7. Utilise Python's file handling operations to read, write, append, and manage files while applying exception handling techniques to handle runtime errors.
8. Integrate programming constructs to design, test, and debug small-scale Python programs that demonstrate logical thinking and structured problem-solving skills.

Working Through This Course

You must study all the units, the suggested texts, and any other pertinent materials to pass this course. In this course, you must eventually turn in the tutor-marked assignments, which are included in each unit, along with some self-assessment tasks. A final test is given after the course. The elements of this course and what you should do are listed below.

Assessment

The evaluation of this course consists of two parts. Tutor-marked assignments come first, followed by the written test. As a result, you are required to remember the information, knowledge, and problem-solving learnt during the course. The coursework must be turned in before the deadline to your tutor. 30% of your final course grade will be determined by the work you submit. You will have to take a final written test after the course. 70% of your final grade will be determined by this exam.

© Copyright 2026 MIVA Open University All Rights Reserved

How to Get The Most From This Course

The study units take the place of university lectures in online learning. One of the major benefits of remote learning is the freedom to read and work through carefully created study materials at your speed, at the time and location that works best for you. Consider it as reading the lecture rather than paying attention to the lecturer. Similar to how a lecturer might assign reading, the study units specify additional books and web links to read.

Each study unit adheres to a standard structure. The first item provides an overview of the unit's subject matter and explains how it fits within the course as a whole as well as how other units relate to it. A list of learning objectives follows this. These goals inform you of the skills you ought to possess after finishing the lesson. Your study should be guided by these learning objectives. Once a unit is complete, you must go back and make sure the goals were met. Your chances of passing the course will considerably increase if you make this a habit.

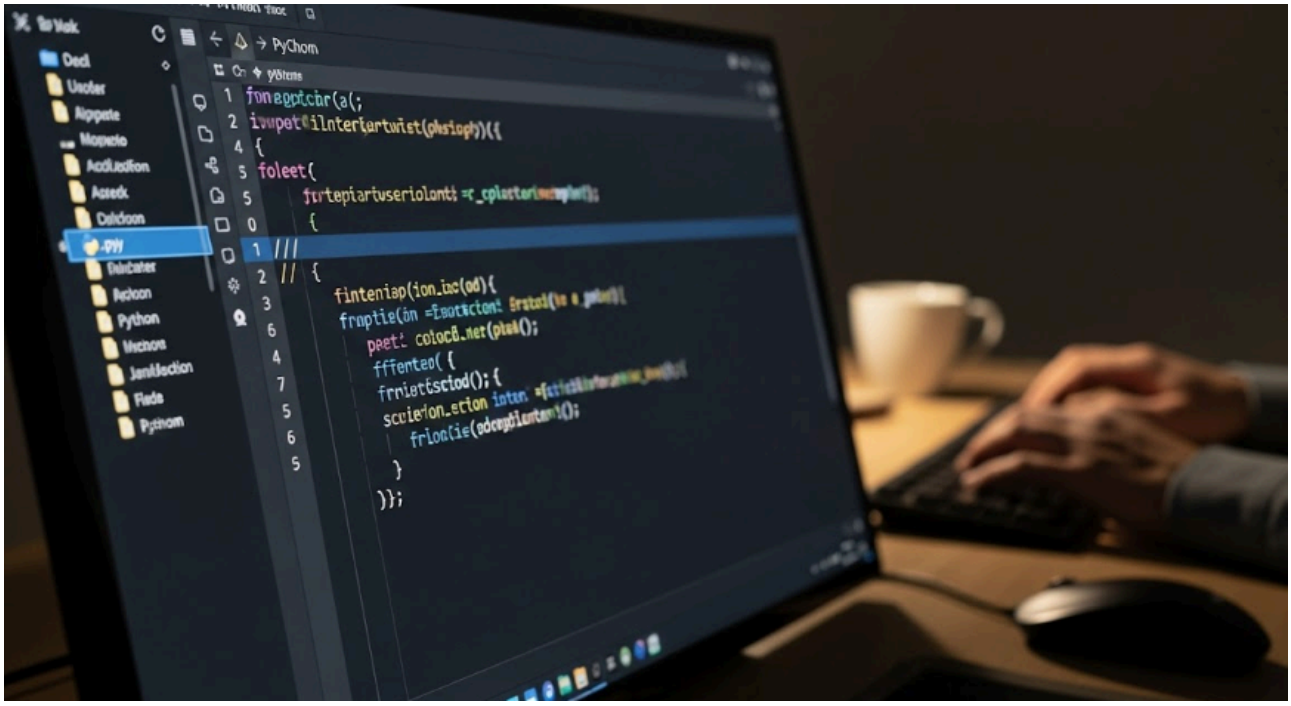
© Copyright 2026 MIVA Open University All Rights Reserved

Study Session One

Introduction to Python

© Copyright 2026 MIVA Open University All Rights Reserved

Introduction



Computer programming is the process of creating code that directs specified activities in a computer, application, or software program. Professionals who write and test code that enables apps and software programmes to run effectively are known as computer programmers. They write a set of instructions directing the computer to operate in a certain manner. Nowadays, computer programs are utilised practically everywhere—in the home, in agriculture, in medicine, in entertainment, in defence, and in communication.

There are three types of computer languages used in passing instructions to the computer, namely high-level Language. Low-Level Language (Assembly Language) and Machine Language.

© Copyright 2026 MIVA Open University All Rights Reserved

Learning Outcomes for Study Session 1

When you have studied this session, you should be able to:

1.1. Define programming and state the main categories of programming languages.

1.2. State the difference between high-level language, low-level Language and machine language.

1.3. Explain the concept of Python.

© Copyright 2026 MIVA Open University All Rights Reserved

1.1. Types of Programming Language

Machine language, assembly language, and high-level language are the three categories of programming languages.

1.1.1. Machine Language

Machine language uses 0s and 1s. While machine language is simpler for computers to grasp, it is more difficult for programmers to comprehend. This is because machine language is the bit-based language of machines. Although it is possible to write programs entirely in machine code, it is more typical to utilise assembly or a high-level language because it is easier to understand and type.

1.1.2. Low-Level Language (Assembly Language)

Understanding assembly language is a little bit simpler. Numbers and English commands are used to replace the machine language bits. An assembler assembles assembly code before the computer executes it. This changes the code back to the 1s and 0s of computer-readable machine language.

1.1.3. High-Level Language

Compared to assembly or machine language, high-level languages use a lot more English commands and are also far more readable. The built-in commands found in many high-level programming languages make it easier for programmers to build loops, create variables with various data types, and work with strings. It should be noted that while all of them are feasible in assembly or machine language, high-level languages greatly simplify their reading, writing, and debugging for the programmer. Scripting languages are some of the most recent

© Copyright 2026 MIVA Open University All Rights Reserved

high-level languages. This indicates that they are not compiled or converted to machine language before the runtime execution of the code. Among the scripting languages are Python, JavaScript, PHP, Ruby, and Bash.

1.2. What is Python?

Python is an interpreted, object-orientated, high-level, dynamically semantic programming language. It is particularly desirable for rapid application development as well as for usage as a scripting or glue language to tie existing components together due to its high-level built-in data structures, dynamic typing, and dynamic binding. Python's straightforward syntax prioritises readability and makes it simple to learn, which lowers the cost of program maintenance. Python's support for sessions and packages promotes the modularity and reuse of code in programs. For all popular platforms, the Python interpreter and the comprehensive standard library are freely distributable and available in source or binary form.

Python programs are simple to debug since a segmentation failure is never caused by a bug or incorrect input. Instead, the interpreter raises an exception when it finds a mistake. The interpreter prints a stack trace if the application doesn't catch the exception. Setting breakpoints, evaluating arbitrary expressions, inspecting local and global variables, stepping through the code one line at a time, and other features are all possible with a source-level debugger. Python's ability to perform self-analysis is demonstrated by the debugger, which is developed in Python.

© Copyright 2026 MIVA Open University All Rights Reserved

1.3. Python vs Other Languages

Python is frequently contrasted with other interpreted languages like Smalltalk, Java, JavaScript, Perl, and Tcl. Also instructive are comparisons to C++, Common Lisp, and Scheme. I'll do a quick comparison of Python to each of these languages in this section. These similarities are limited to matters of language. In reality, additional practical considerations like cost, availability, skill level, prior investment, or even sentimentality frequently determine the choice of a programming language.

1.3.1. Python vs. Java

Although Python applications typically run more slowly than Java ones, they also require a lot less time to develop. Compared to equivalent Java programs, Python programs are typically three to five times shorter. Python's dynamic typing and built-in high-level data types are responsible for this distinction. A Python programmer, for instance, doesn't waste any time specifying the kinds of arguments or variables, and Python's robust polymorphic list and dictionary types, for which the language comes with comprehensive syntactic support, are used in practically every Python application.

1.3.2. Python Vs JavaScript

The "object-based" portion of Python is comparable to JavaScript. Python allows a programming approach that employs basic functions and variables without using class definitions, similar to JavaScript (but unlike Java). However, that is all that exists for JavaScript. Furthermore, Python's genuine object-oriented programming approach, where classes and inheritance play a significant role,

© Copyright 2026 MIVA Open University All Rights Reserved

enabling the creation of considerably larger programmes and improved code reuse.

1.3.3. Python Vs Perl

Both Python and Perl have a similar history—Unix scripting, which both languages have since outgrown—and share many features. However, their philosophies are very different. Regular expressions, file scanning, and report-producing capabilities are just a few of the features incorporated into Perl that emphasise support for basic application-orientated activities. Python emphasises supporting common programming paradigms like object-orientated programming and data structure design, and it also encourages programmers to produce understandable (and thus maintainable) code by offering an elegant but not unduly esoteric syntax. As a result, while Python rarely surpasses Perl in its initial application domain, it has a much wider range of uses than Perl does.

1.3.4. Python Vs C++

Almost everything said about Java also applies to C++, but more so: Python code is sometimes 5–10 times shorter than identical C++ code, but Java code is typically 3–5 times shorter! According to anecdotal evidence, one Python programmer can accomplish in two months what it would take two C++ programmers a year to perform. Python excels when used as a glueing language to join C++ components.

© Copyright 2026 MIVA Open University All Rights Reserved

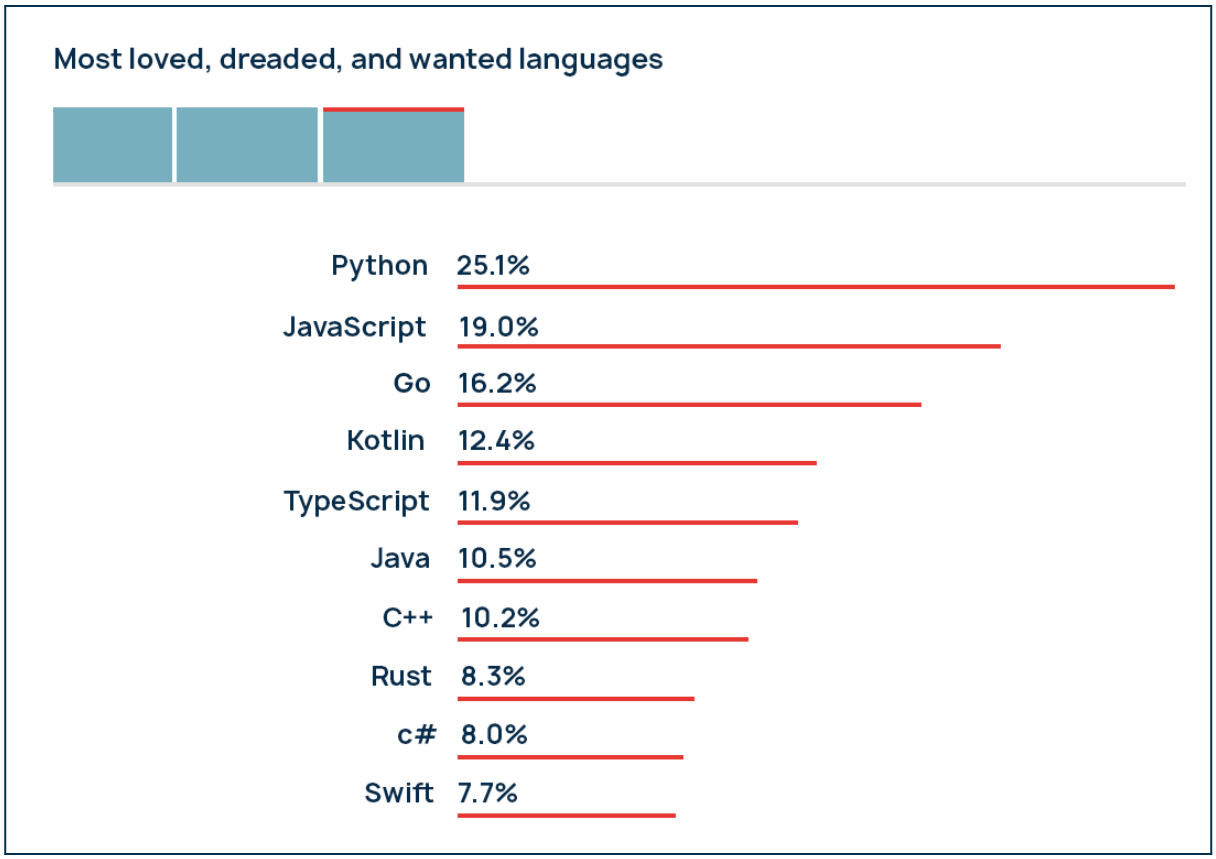


Figure 1.1 Stack overflow developer survey 2018

In-text Questions and Answers (ITQs and ITAs)

ITQ 1.1: What are the three main categories of programming languages?

Answer: The three main categories of programming languages are machine language, low-level language (assembly language), and high-level language.

ITQ 1.2: How does Python differ from Java in terms of development time?

Answer: Python typically requires less time to develop compared to Java. Python programs are usually three to five times shorter than equivalent Java programs, thanks to its dynamic typing and built-in high-level data types.

© Copyright 2026 MIVA Open University All Rights Reserved

1.4. Installation of Python

You need an editor to type the program into while writing Python programs. Having an editor specifically made for writing Python scripts is useful. An IDE, or integrated development environment, is a type of editor created especially for developing programs. An IDE is not merely an editor. As you develop a programme, it offers highlighting and indentation that can be useful.

This section presents how Python can be installed. Two approaches to installing Python are presented below:

1. **Method 1**

According to the instructions below, download and install Python from python.org on different operating systems.

After downloading, double-click the .exe (for Windows) or .pkg (for Mac) file to begin the installation of Python by following the on-screen directions.

Use the command below at the prompt to see if Python is already installed on the Linux OS.

```
$ python –version. ...
```

Use the distribution package management to install Python if version 2.7 or later isn't already there. The names of the command and package can differ.

Use apt on Debian-based variants like Ubuntu.

```
$ sudo apt-get install Python 3
```

To check that Python is properly installed, open the command prompt and issue the following command:

```
$ python3 –version
```

© Copyright 2026 MIVA Open University All Rights Reserved

```
Python 3.6.2
```

Similar to this, we may use installers like pip to download and install required libraries like NumPy, Matplotlib, etc., independently. You can use the commands listed below for this purpose.

```
$pip install numpy
$ pip install matplotlib
$pip install pandas
$pip install seaborn
```

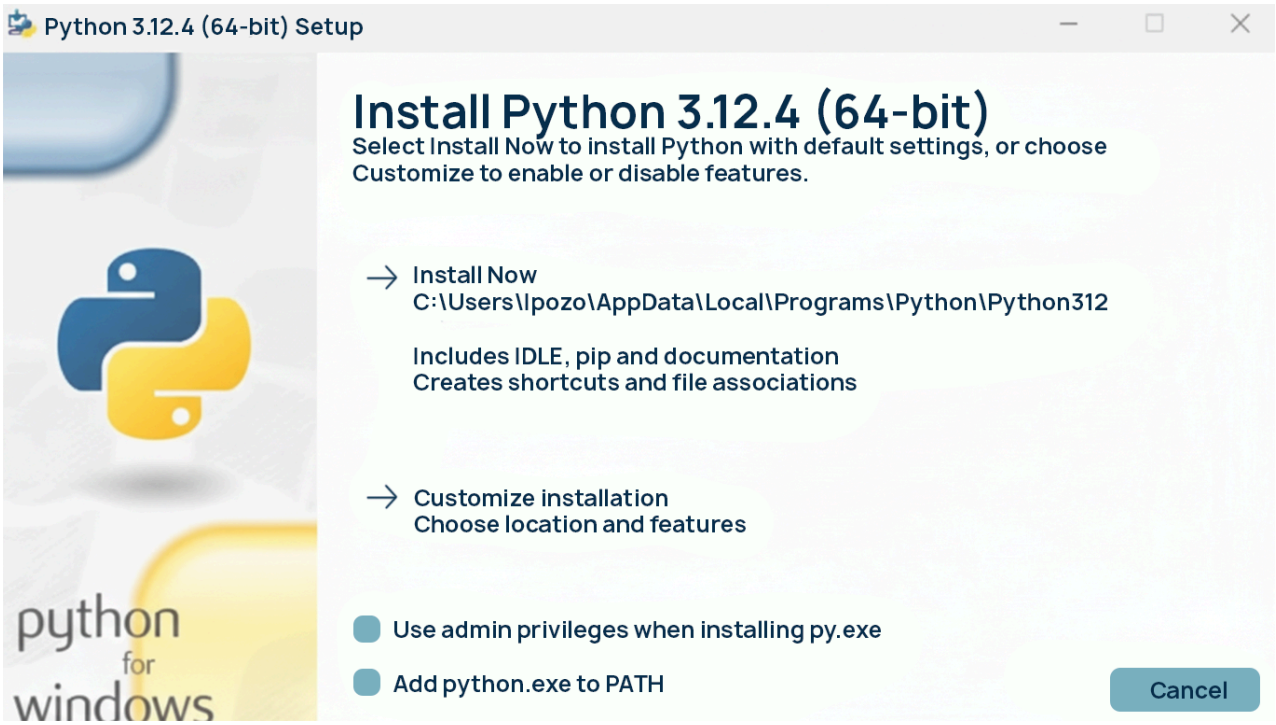


Figure 1.2: Python installation interface

2. **Method 2**

© Copyright 2026 MIVA Open University All Rights Reserved

Alternatively, we can install the Anaconda distribution to simultaneously install Python and additional scientific computing and machine learning packages. It consists of many machine learning tools, including NumPy, scikit-learn, and Matplotlib, and there is a Python version for Linux, Windows, and OS X. Jupyter Notebook, an interactive Python environment, is also included. Depending on our needs, we can install Python 2.7 or any 3.x version. The installation procedure may take up to 15-20 minutes. Choose the installation procedure described here, depending on your operating system.

For Windows, check in the column with Python 2.7 or 3.x under the Anaconda for Windows section. There are two installers available, one for 64-bit Windows and the other for 32-bit Windows. Pick the appropriate one.

Scroll down to the Anaconda for OS X section for Mac OS. Look for Python 2.7 or 3.x in the column. Keep in mind that there is only one installer version available here—the 64-bit version.

You can choose the "Anaconda for Linux" part for the Linux OS. Look for Python 2.7 or 3.x in the column.

Keep in mind that if there are any other Python installations on your system, you must make sure that Anaconda's installation of Python affects only one directory.

© Copyright 2026 MIVA Open University All Rights Reserved

Conclusion

In this session, we introduced students to programming, including its definition and the various types of programming languages: machine, low-level, and high-level. We also explored the programming language chosen for this course, Computer Programming 1, highlighting its advantages over other languages. Additionally, a dedicated sub-section guided students through the installation process of Python for practical application.

Summary

1. Programming, also known as coding, involves creating instructions for devices using programming languages like Python. These languages come in three types: machine language (using 0s and 1s), assembly language (using mnemonics), and high-level language (a mix of English-like and mathematical notations).
2. Python, a widely used and beginner-friendly programming language, is utilised for various applications such as software development, website creation, automation, and data analysis. Its versatility and ease of use have made it one of the most popular languages among developers, as evidenced by the 2018 Stack Overflow Developer Survey. This unit includes instructions for installing Python on computers.

© Copyright 2026 MIVA Open University All Rights Reserved

Self-Assessment Questions and Answers (SAQs and SAAs)

SAQ 1.1: Define 'programming' and state the three main categories of programming languages.

Answer: Programming is the process of writing instructions that a computer can execute. The three main categories of programming languages are machine language, low-level language (assembly), and high-level language.

SAQ 1.2: Discuss the main difference between machine language and high-level language.

Answer: Machine language consists of binary code (0s and 1s), which is difficult for humans to understand but easy for computers to execute. High-level languages use more readable commands (like English), making them easier for programmers to write, read, and debug.

SAQ 1.3: How does Python support object-oriented programming?

Answer: Python supports object-oriented programming (OOP) by allowing programmers to define classes and objects, enabling inheritance and polymorphism. Python's OOP approach allows for the creation of modular, reusable, and maintainable code.

Multiple-Choice Questions and Answers (MCQs and MCAs)

1. **Which of the following is a characteristic of machine language?**

   A. It is easy for programmers to understand.

   B. It uses 0s and 1s to instruct the computer.

© Copyright 2026 MIVA Open University All Rights Reserved

C. It is human-readable.

D. It is a high-level programming language.

Answer: B

Explanation: Machine language is composed of binary code (0s and 1s) that computers can understand directly, but it is not easily understood by humans.

2. **Which of the following is a reason why Python is considered beginner-friendly?**

A. It requires advanced knowledge of computer architecture.

B. It uses complex syntax and structures.

C. It has a simple and readable syntax.

D. It is compiled before execution.

Answer: C

Explanation: Python's syntax is designed to be simple and readable, making it easier for beginners to learn.

3. **Which of the following is NOT a type of programming language?**

A. High-level language

B. Low-level language

C. Machine language

D. Graphical programming language

Answer: D

Explanation: High-level, low-level, and machine languages are the three main categories. Graphical programming languages are not a standard category.

© Copyright 2026 MIVA Open University All Rights Reserved

4. **What is the role of Python's interpreter?**

   A. It compiles the code into machine language.

   B. It reads the code and executes it line by line.

   C. It translates the code into Java.

   D. It stores the code in a binary format.

   Answer: B

   Explanation: Python's interpreter reads the Python code and executes it directly, which makes debugging easier.

5. **How does Python handle errors during program execution?**

   A. It crashes the program immediately.

   B. It ignores the errors.

   C. It raises an exception and prints a stack trace.

   D. It halts the program silently.

   Answer: C

   Explanation: Python's interpreter raises exceptions when it encounters errors and prints a detailed stack trace to help with debugging.

Tutor-Marked Assessments (TMAs)

1. Compare Python and Java in terms of development time and efficiency, explaining why Python is often the preferred choice for rapid application development.
2. How does Python's object-oriented programming approach contribute to its versatility? Provide examples to support your argument.

© Copyright 2026 MIVA Open University All Rights Reserved

3. Analyse the installation process for Python on different operating systems, highlighting key steps and potential challenges.
4. Assess the role of Python in the modern software development landscape, comparing it with other languages like C++ and Java. Explain why Python's popularity continues to grow.

© Copyright 2026 MIVA Open University All Rights Reserved

Study Session Two

Python Program Fundamentals

© Copyright 2026 MIVA Open University All Rights Reserved

Introduction

The ways to input and output data in Python are presented in this unit. Since a computer follows the process of garbage in and garbage out, we illustrate with examples how to input data and print out results.

Before we can work with input and output, however, we must first understand the fundamental building blocks of any Python program. Just as a house is built from bricks, mortar, wood, and wiring, a Python program is built from basic elements that give it structure and meaning. These elements include comments that help us document our code, variables that store information, data types that define what kind of information we are working with, and operators that allow us to manipulate that information.

In this session, we will start by learning how to execute Python programs in your development environment. From there, we will explore how to write clear and maintainable code using comments. The core of the session will focus on Python's built-in data types—such as numbers, strings, and booleans—and the operators we can use with them. Understanding these fundamentals is essential before we can effectively handle user input and generate meaningful output. By the end of this session, you will have a solid foundation in Python's basic syntax and be ready to build simple but functional programs.

Learning Outcomes for Study Session 2

When you have studied this session, you should be able to:

2.1. Execute Python programs within the installed programming environment.

© Copyright 2026 MIVA Open University All Rights Reserved

2.2. Apply single-line and multi-line comments effectively to document Python code.

2.3. Identify and differentiate between various data types and operators used in Python programming.

2.4. Demonstrate the use of Python's built-in data types by writing simple programs that store and manipulate different kinds of data.

2.5. Apply arithmetic, comparison, and logical operators to solve basic computational problems.

© Copyright 2026 MIVA Open University All Rights Reserved

2.1. Input and Output Statements

2.1.1. Input statement syntax and examples

Syntax

input (system prompt)

Parameter Values:

Parameter: System prompt

Description: A string that represents a default message before the input.

Example

```
x = input("My name is John")
print("Hello, " + x)
```

2.1.2. Output statement syntax and examples

Syntax

print(object(s), sep=separator, end=end, file=file, flush=flush)

object(s): Any item you choose, in any quantity. Before printing, it will be transformed into a string.

sep='separator': Optional. If there are multiple objects, describe how to separate them. Default is ' '

end='end': Optional. Specify what to print at the end. Default is '\n' (line feed)

file: Optional. An object with a write method. Default is sys.stdout

Flush: Optional. A Boolean specifies if the output is flushed (True) or buffered (False). Default is False

Example

```
print("Hello", "I am watching football!")
```

© Copyright 2026 MIVA Open University All Rights Reserved

```
x = ("Goat", "Rabbit", "Pig")
print(x)
```

2.2. Comments, Data types and Operators

In continuation of how to write a Python program, comments, data types, and operators, which form the key features, are presented in this unit. Comments are needed for the readability and understandability of a program, while data types and operators are needed for the manipulation of data.

2.2.1. Comments

Python comments start with the hash symbol # and continue to the end of the line. However, hash characters in a string are not regarded as comments. A comment can be written on a distinct line, next to the relevant line of code, or in a block of multiple lines.

Writing comments is useful in many different ways in Python. Among the important uses are improved readability and describing the code to other people, and it enables one to understand his/her code after a long time of coding it.

Advantages of Using Comments in Python

Python comments offer a number of benefits. Their main advantages are:

a. Enables other programmers to comprehend the code with ease.
b. The code then makes sense on its own.
c. Aids in recalling the rationale for the usage of a particular command, method, or function in the code.
d. Enables the interpreter to disregard certain code portions when doing tests.

© Copyright 2026 MIVA Open University All Rights Reserved

Types of Comments in Python

Single-line, multi-line, and docstring comments are the three different types of comments. Depending on the kind, comments have different syntaxes. Each type of comment will be covered individually in this tutorial along with examples.

Single-Line Comments

The "#" symbol denotes the start of a single-line comment. After the hash symbol (#), anything written on a single line is regarded as a comment. The format for single-line comments is as follows:

```
# Comment here
```

In Python, there are two ways to use single-line comments. It can be used either before or after the code. The example given below demonstrates how comments can be used both ways.

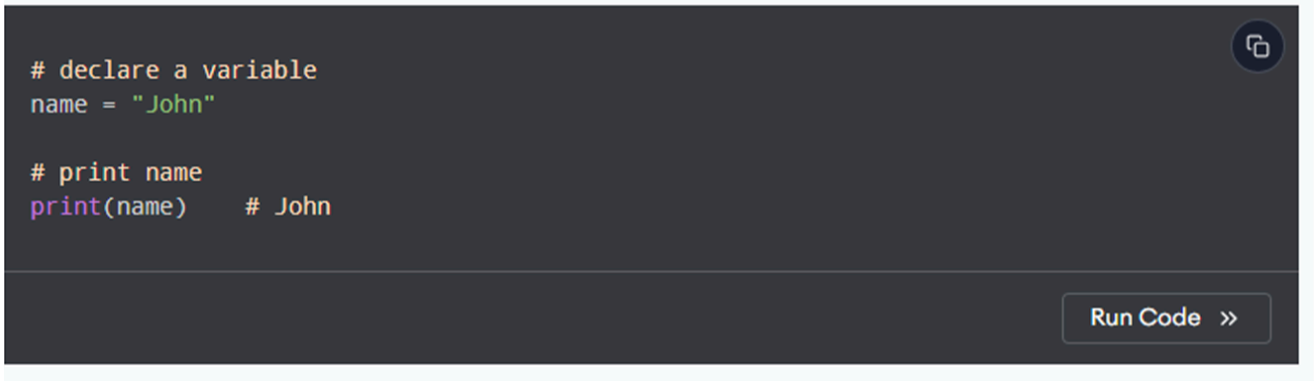


Figure 2.1 Single-line comments

Multi-Line Comments

Multi-line comments are not supported by Python. There are numerous solutions to this problem, though. Although none of these approaches are technically multi-line comments, you can nevertheless use them as such. The first process includes beginning each line of the remark with a #.

© Copyright 2026 MIVA Open University All Rights Reserved



Figure 2.2 Approach 1—Starting each line with a #

The second method involves using string literals without putting them in any variables. The Python interpreter skips string literals that aren't assigned to variables. Take advantage of this to write comments with many lines. You have the option of using a single (') or double ("") quotation.

© Copyright 2026 MIVA Open University All Rights Reserved

```
'Single-line comments using string literals'

""" Python program to demonstrate
 multiline comments"""
print("Multiline comments")
```

Figure 2.3 Approach 2—Using string Literals

In-text Questions and Answers (ITQs and ITAs)

ITQ 2.1: What is the syntax of the Python input statement, and how is it used in practice?

Answer: The syntax of the Python input statement is input("prompt"). It displays a prompt message to the user and waits for their input, which is then returned as a string. Example:

x = input("My name is John")
print("Hello, " + x)

ITQ 2.2: What is the difference between single-line and multi-line comments in Python?

Answer: Single-line comments in Python begin with the # symbol. Multi-line comments can be achieved by starting each line with # or using unassigned string literals. For docstrings, triple quotes (""" or ''') are used.

© Copyright 2026 MIVA Open University All Rights Reserved

Python Docstrings

To comment on sessions, methods, functions, objects, and classes, Python has an internal tool called docstrings. After creating a session, function, method, etc., they are written in the first line using three quotation marks (" or ""). The interpreter will not recognise it as a docstring if you do not use it in the first line. Additionally, you can access docstrings by utilising the ___doc___ attribute.

```
def my_func():
    '''This is a docstring using triple single quotes.'''
    return None

print("Using __doc__:")
print(my_func.__doc__)

print("Using help():")
help(my_func)
```

```
Using __doc__:
This is a docstring using triple single quotes.
Using help():
Help on function my_func in module __main__:

my_func()
    This is a docstring using triple single quotes.
```

Figure 2.4: Python Docstrings

A program's comments are an essential component. Therefore, learning how to make effective comments is crucial. Here are a few qualities that Make for good comments: Make sure they are brief; avoid writing generic comments unless they offer information; describe a function or method's broad task rather than its minute details in your comments; good comments are self-explanatory; and avoid writing repetitive comments.

© Copyright 2026 MIVA Open University All Rights Reserved

2.2.2. Data Type

Data types are used for performing mathematical operations. There are various data types used in python. The data types and their descriptions are presented in the table below:

Table 2.1: Data Types and their Descriptions

Data Types	How to use them
Numeric data types	int, long, float, complex
String data types	str
Sequence types	list, tuple, range
Binary types	bytes, bytearray, memoryview
Mapping data type	dict
Boolean type	bool
Set data types	set, frozenset

© Copyright 2026 MIVA Open University All Rights Reserved

a. **Numeric Data Type:**

Int holds a non-limited-length signed integer, long holds a long integer, float houses floating precision numbers, and complex is for complex numbers.

Example:

```
# Create a variable with an integer value.
x=200
print("The variable
```

```
   Create a variable with a float value.
y=12.2345
print("The variable type for ", y, " is ", type(y))
```

```
# Create a variable with a complex value.
z=200+3j
print("The variable
```

b. **String Data Type**

These are a sequence of characters. They are represented by a double or single quote.

```
a = "string in a double quote"
b = 'string in a single quote'
print(a)
print(b)
```

© Copyright 2026 MIVA Open University All Rights Reserved

c. **List Data Types**

The list is a flexible data type that is unique to Python. Python's list is interesting because it can store multiple sorts of data at once.

```
#list of having only integers
a = [1, 2, 3, 4, 5, 6]
print(a)

#list of having only strings
b=["Ter","Bob","Vivian"]
print(b)
```

d. **Tuple**

The tuple is an additional data type that is a list-like collection of data. But it cannot be altered. In other words, a tuple's data is write-protected. The use of parentheses and commas is required when expressing data in a tuple.

Example:

```
#tuple having only integer type of data.
a=(1,2,3,4)
print(a) #prints the whole tuple

#tuple hav1, 2, 3, goiple types of data.
b=("hello", 1,2,3,"go")
print(b) #prints the whole tuple
```

© Copyright 2026 MIVA Open University All Rights Reserved

e. **Dictionary**

A key-value pair-formatted unordered series of data is a Python dictionary. It resembles the hash table kind. Dictionary entries take the key:value format and are enclosed in curly brackets. Among a large amount of data, it is very helpful to retrieve information in an optimised way.

Example:

```
#a sample dictionary variable

a = {1: "first name , 2: last name", "age: 33}

#print value having key=1
print(a[1])
#print value having key=2
print(a[2])
#print value having key="age"
print(a["age"])
```

2.2.3. Operators

Operators are used to carry out operations on variables and values. Operators are divided in Python into the following groups: Assignment operators, Arithmetic operators, Logical operators, Comparison operators, Identity operators, Membership operators, and bitwise operators

a. **Assignment operators**: They are used to assign values to variables.

© Copyright 2026 MIVA Open University All Rights Reserved

Table 2.2: Assigned Operators

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

b. **Arithmetic operators:** Used for mathematical operations

© Copyright 2026 MIVA Open University All Rights Reserved

Table 2.3: Arithmetic Operators

Operator,	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

c. ***Logical operators:*** Used for combining conditional statements.

Table 2.4: Logical Operators

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result and return False if the result is true	not (x < 5 and x < 10)

d. **Comparison operators:** Used for comparing two values

© Copyright 2026 MIVA Open University All Rights Reserved

Table 2.5: Comparison Operators

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

e. **Identity operators:** Identity operators are used to compare objects to determine whether they are indeed the same object in the same memory address rather than whether they are equal.

Table 2.6: Identity Operators

Operator	Description	Example
is	Returns True if both variables are the same object	x is y.
is not	Returns True if both variables are not the same object	x is not y.

© Copyright 2026 MIVA Open University All Rights Reserved

f. **Membership operators:** For testing, if a sequence is presented in an object

Table 2.7: Membership Operators

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

g. **Bitwise operators:** Used for comparing binary numbers

Table 2.8: Bitwise Operators

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off

© Copyright 2026 MIVA Open University All Rights Reserved

>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left and let the rightmost bits fall off

2.3 General Examples

***Example 1**: A simple program that displays "Hello, World!"*

```
# This program prints Hello, world!

print('Hello, world!')
```

*Example 2: **Program to Add Two Numbers***

```
# This program adds two numbers

number1 = 1.5
number2 = 6.3

# Add two numbers
sum = number1 + number2

# Display the sum
print('The sum of {0} and {1} is {2}'. format(number1, number2, sum))
```

© Copyright 2026 MIVA Open University All Rights Reserved

Example 3: *Add Two Numbers With User Input*

```
# Store input numbers
num1 = input ('Enter first number: ')
num2 = input ('Enter second number: ')

# Add two numbers
sum = float(num1) + float(num2)

# Display the sum
print('The sum of {0} and {1} is {2}'. format(num1, num2, sum))
```

Example 4: *Program to Find the Square Root*

```
num = 8

# To take the input from the user
#num = float(input('Enter a number: '))

num_sqrt = num ** 0.5
print('The square root of %0.3f is %0.3f' % (num, num_sqrt))
```

Example 5: *Program to Calculate the Area of a Triangle*

```
# Python Program to find the area of a triangle

a = 5
```

© Copyright 2026 MIVA Open University All Rights Reserved

```
b = 6
c = 7

# Uncomment below to take inputs from the user
# a = float(input('Enter first side: '))
# b = float(input('Enter second side: '))
# c = float(input('Enter third side: '))

# calculate the semi-perimeter
s = (a + b + c) / 2

# calculate the area
area = (s*(s-a)*(s-b)*(s-c)) ** 0.5
print('The area of the triangle is %0.2f' % area)
```

Example 6: *Programme to calculate $a^2 + b^2 + c^2$*

```
print "value of a:"
a=input()
print "value of b:"
b=input()
print "value of c:"
c=input()
d=(a*a)+(b*b)+(c*c)
```

© Copyright 2026 MIVA Open University All Rights Reserved

```
print(d)
```

Conclusion

Python's fundamental features, including the input() and print() functions for user interaction, alongside its excellent commenting system, facilitate code readability and comprehension.

Understanding data types and operators is crucial not only in Python but also in other programming languages for efficient coding, especially for mathematical operations.

Summary

1. The input and output statements in Python are described in this unit with appropriate syntax and examples. Students can start practising with the given examples.
2. In the course of writing a program, comments, data and operators are used. This unit explains why comments are needed in a program and the types of comments in Python. Examples are presented to further explain them. Also, data types used in Python are explained. Although Python does not declare data types like other programming languages, it is necessary to know how to use the data types in Python. The unit concludes with an explanation of the types of operators used in Python.

© Copyright 2026 MIVA Open University All Rights Reserved

Self-Assessment Questions and Answers (SAQs and SAAs)

SAQ 2.1: What are the primary data types used in Python for mathematical operations?

Answer: The primary data types for mathematical operations in Python are int, float, and complex. These data types allow for various mathematical manipulations and calculations.

SAQ 2.2: How can multi-line comments be implemented in Python?

Answer: Multi-line comments in Python can be simulated by using # at the start of each line or by using string literals without assigning them to a variable. Docstrings (triple quotes) can also be used for multi-line documentation.

SAQ 2.3: Discuss the role of operators in Python programming.

Answer: Operators in Python are used to perform operations on variables and values. These include arithmetic, comparison, logical, assignment, identity, membership, and bitwise operators, each serving a specific purpose in manipulating data.

Multiple-Choice Questions and Answers (MCQs and MCAs)

1. **What is the purpose of the input() function in Python?**
   A. To print data to the console.
   B. To accept user input.
   C. To assign values to variables.
   D. To compare two values.

© Copyright 2026 MIVA Open University All Rights Reserved

Answer: B

Explanation: The input() function prompts the user for input and returns the data entered as a string.

2. **Which of the following is NOT a valid data type in Python?**

   A. str

   B. int

   C. float

   D. real

   Answer: D

   Explanation: Real is not a data type in Python. The valid types are int, str, float, etc.

3. **Which operator is used for integer division in Python?**

   A. %

   B. /

   C. //

   D. **

   Answer: C

   Explanation: The // operator in Python performs floor division, which divides and returns the integer quotient.

4. **Which of the following is a valid way to write a single-line comment in Python?**

   A. /* This is a comment */

   B. # This is a comment

© Copyright 2026 MIVA Open University All Rights Reserved

C. – This is a comment

D. ' This is a comment.'

Answer: B

Explanation: In Python, comments begin with the # symbol.

5. **What is the output of the following code?**

```
x = "Python"
print(x[1])
```

A. P

B. y

C. t

D. h

Answer: B

Explanation: The index 1 refers to the second character in the string, which is 'y'.

Tutor-Marked Assessments (TMAs)

1. How do Python's data types differ from those in statically typed languages? Discuss how Python handles data types during runtime.
2. Compare and contrast the arithmetic operators and logical operators in Python. Provide real-world examples of when each would be used.
3. Python allows for multi-line comments using string literals. Discuss the pros and cons of this approach compared to traditional multi-line comment syntax in other languages.

© Copyright 2026 MIVA Open University All Rights Reserved

4. Write a Python program that takes two numbers as input and calculates the sum, difference, product, and quotient. Use appropriate comments and explain the program's output.

© Copyright 2026 MIVA Open University All Rights Reserved

Study Session Three

Decision and Control

© Copyright 2026 MIVA Open University All Rights Reserved

Introduction

A decision is the specification of actions to be taken in response to conditions that may arise during program execution. Multiple expressions are evaluated by decision structures, and the results are TRUE or FALSE. If the outcome is TRUE or FALSE, you must decide which statement to run and which action to take. This fundamental concept of making choices is what gives programs their intelligence and adaptability. Without decision-making capabilities, a program would simply execute the same sequence of instructions every time, regardless of the input or context.

In this session, we will explore how to control the flow of your Python programs using two essential tools: decision structures and loops. Decision structures, such as IF-Else statements, allow your program to evaluate conditions and choose different paths of execution based on the result. This is how a program can, for example, check if a user's password is correct and either grant access or display an error message. Loops, on the other hand, enable your program to repeat blocks of code efficiently. Instead of writing the same instruction one hundred times, you can write a loop that repeats a block of code one hundred times. This is essential for tasks like processing items in a list, reading data from a file, or waiting for user input. By the end of this session, you will be able to write programs that can make decisions, repeat actions, and respond intelligently to different situations, moving beyond simple linear scripts to create more dynamic and powerful applications.

© Copyright 2026 MIVA Open University All Rights Reserved

Learning Outcomes for Study Session 3

When you have studied this session, you should be able to:

3.1. Explain how to use decision statements like the IF-Else and the nested IF statement in writing programmes.

3.2. Develop the ability to construct and implement loops in Python, including the FOR loop and WHILE loop, to efficiently iterate over sequences of data or perform repetitive tasks.

3.3. Develop critical thinking skills by evaluating different approaches to decision-making and loop implementation in Python, considering factors such as efficiency, readability, and code maintainability, to make informed programming choices.

3.4. Identify and correct common syntax and logical errors in decision and loop structures.

3.5. Choose the appropriate type of loop (FOR or WHILE) for a given programming task.

© Copyright 2026 MIVA Open University All Rights Reserved

3.1. Decision Statement

3.1.1. IF – Else Statement

When we wish to only run a piece of code if a certain condition is met, we must make use of a decision statement. Making decisions is useful when what we want to do depends on user input or another value that is unknown at the time our program is written. This is frequently the case, and Python, like other useful programming languages, has the capability of comparing data and then acting accordingly. It gives a direction of the program control as depicted in the diagram below:

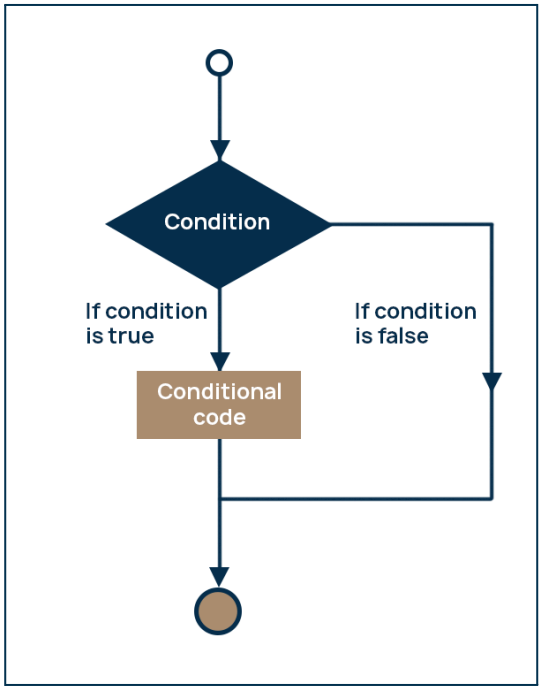


Figure 3.1 IF - Else Statement

The IF statement has a boolean expression followed by one or more statements, while the IF-ELSE statement has an alternative statement that executes when the boolean expression is FALSE.

© Copyright 2026 MIVA Open University All Rights Reserved

The syntax for the IF statement is given below:

```
if expression:
  statement(s)
```

```
# Example
var1 = 100
if var1:
  print ("Got a true expression value")
  print var1
```

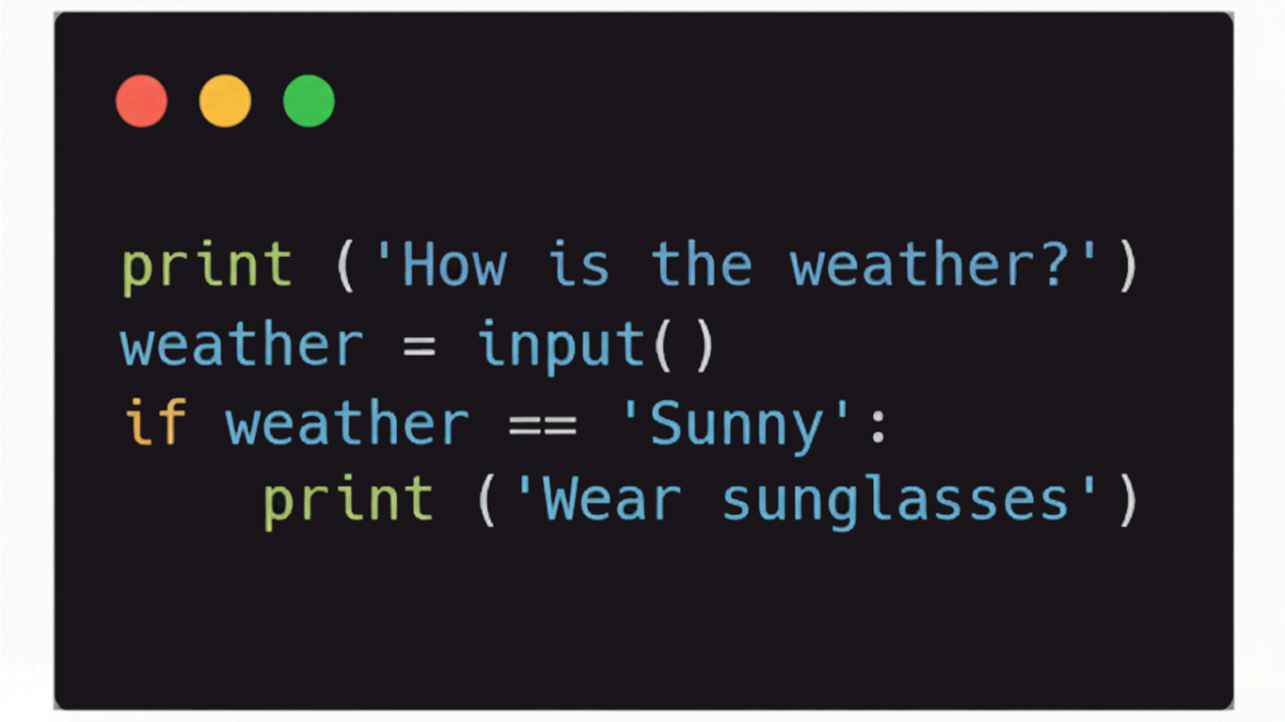


Figure 3.2 IF Syntax

© Copyright 2026 MIVA Open University All Rights Reserved

The syntax of the *IF...ELSE* statement is given below:

```
if expression:
  statement(s)
else:
  statement(s)
```

```
Example
x = 100
if x:
  print ("Got a true expression value")
  print x
else:
  print ("Got a false expression value")
  print x
```

3.1.2. Nested IF Statement

When a condition resolves to true, there may be instances where you want to check for another condition. You can use the nested if construct in this circumstance.

```
The syntax of the nested if...else construct may be –
if expression1:
  statement(s)
  if expression2:
    statement(s)
  elif expression3:
```

© Copyright 2026 MIVA Open University All Rights Reserved

```
    statement(s)
  elif expression4:
    statement(s)
  else:
    statement(s)
else:
  statement(s)
```

Example

```
var = 100
if var < 200:
  print " value is less than 200"
  if var == 150:
    print "value is 150"
  elif var == 100:
    print "value is 100"
  elif var == 50:
    print "value is 50"
  elif var < 50:
    print "value is less than 50"
else:
  print "Could not find true value."

print "Thank you and goodbye!"
```

© Copyright 2026 MIVA Open University All Rights Reserved

In-text Questions and Answers (ITQs and ITAs)

ITQ 3.1: What is the basic syntax of an IF-Else statement in Python?

Answer: The basic syntax of an IF-Else statement in Python is:

```
if expression:
  statement(s)
else:
  statement(s)
```

ITQ 3.2: What is a nested IF statement, and how does it work?

Answer: A nested IF statement is an IF statement inside another IF statement, which allows for further condition checks if the outer condition is true. **Example:**

```
var = 100
if var < 200:
  print("value is less than 200")
  if var == 150:
    print("value is 150")
```

3.2. Loops, Range, Break And Continue

Statements are typically executed in order: the first statement in a function is executed first, then the second, and so on. You might need to run a piece of code more than once in some circumstances. Programming languages offer a range of control structures that enable more intricate execution pathways. A loop statement enables us to repeatedly run a statement or set of statements.

© Copyright 2026 MIVA Open University All Rights Reserved

3.2.1. FOR Loop

In a FOR loop, a sequence's expression list is evaluated first if one is present. The iterating variable, *iterating_var, is* then set to contain the first element of the sequence. The statements block is then put to use. The statement(s) block is executed up until the end of the sequence, and each item in the list is assigned to *iterating_var*. The picture below depicts the flow:

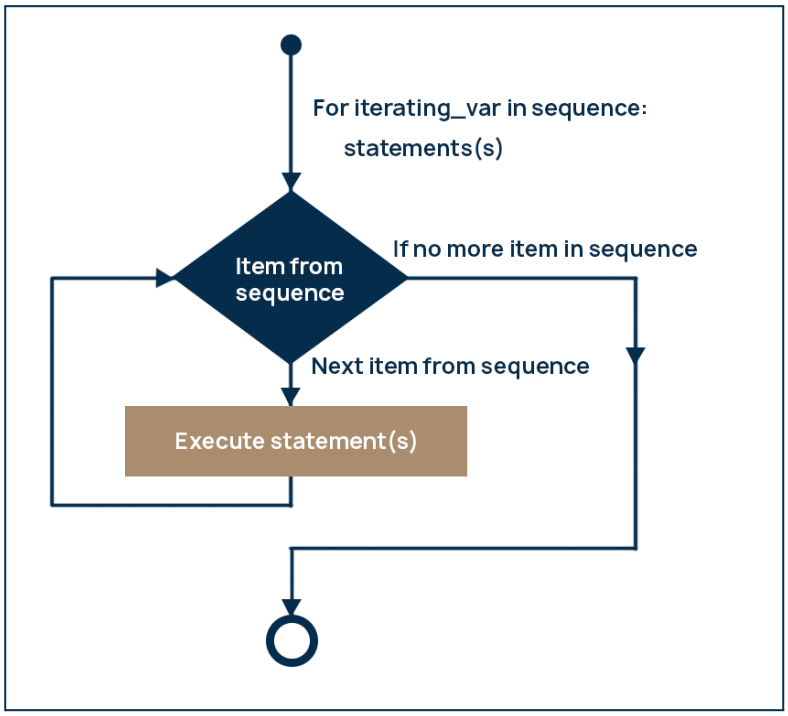


Figure 3.3: FOR Loop

The syntax for the FOR loop is given thus:

```
for iterating_var in sequence:
  statement(s)
```

```
# Prints out the numbers 0, 1, 2, 3, and 4
for x in range(5):
  print(x)
```

© Copyright 2026 MIVA Open University All Rights Reserved

```
# Prints out 3, 4, 5
for x in range (3, 6):
  print(x)

# Prints out 3, 5, 7
for x in range (3, 8, 2):
  print(x)
```

3.2.2. WHILE Loop

A while loop statement in the Python programming language repeatedly executes a target statement as long as a given condition is true. A block of statements or a single statement may be considered a statement in this context. Any expression that is true for the condition must not be zero. As long as the circumstance is true, the loop iterates. Program control moves to the line immediately after the loop when the condition is false.

After a programming construct, any statements that are indented by the same number of character spaces are regarded as belonging to a single block of code in Python. The way that Python groups statements is by indentation. The flow diagram is given below:

© Copyright 2026 MIVA Open University All Rights Reserved

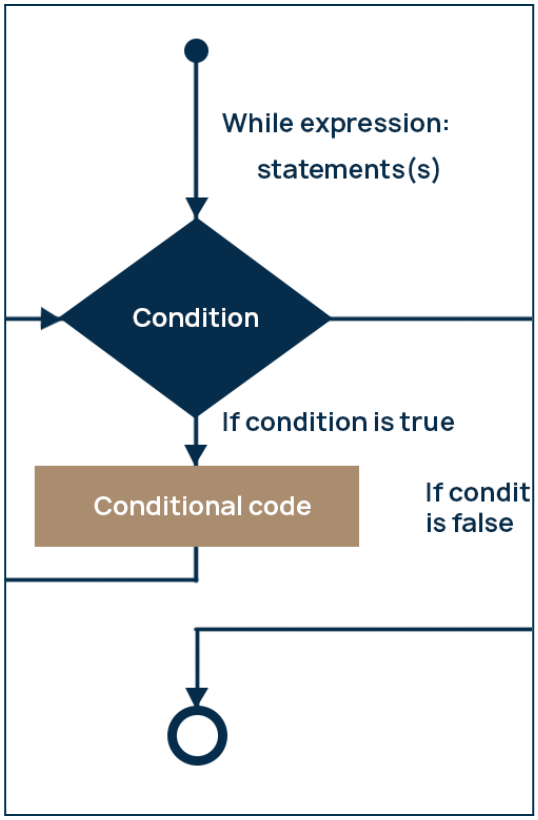


Figure 3.4 While loop

The syntax of a **while** loop in the Python programming language is –

```
while expression:
  statement(s)
```

The while loop may never run. The while loop's body is skipped and the first line after it is performed when the condition is tested and the result is false.

```
Example
count = 0
while (count < 10):
  print 'The count is:', count
  count = count + 1
print "Goodbye!"
```

© Copyright 2026 MIVA Open University All Rights Reserved

The print and increment statements in this block are periodically executed until the count is no longer less than 10. The index count is displayed at its current value and then raised by 1 after each repetition.

3.2.3 Range, Break and Continue

a. Range

The range() function returns a series of numbers that, by default, starts at 0 and increments by 1 before stopping before a given number. The syntax is given below:

range(start, stop, step)

Table 3.1: Range Parameters And Descriptions

Parameter	Description
start	Optional. An integer number that shows the starting point. Default is 0
stop	Required. An integer number that shows the stopping point.
step	Optional. An integer number for increment. Default is 1

© Copyright 2026 MIVA Open University All Rights Reserved

Example: Create and print in sequence numbers from 4 to 6:

```
x = range(4, 7)
for i in x:
 print(i)
```



Figure 3.5: range() function code example

b. Break and Continue

You'll see in this section how the break and continue statements may be used to halt the execution of a for loop. While 'continue' only ends the current iteration of the loop, 'break' terminates the loop entirely.

When an external condition is met, you might have to totally exit a loop. Alternatively, you might want to bypass a section of the loop and move on to the next execution. Python has statements called 'break' and 'continue' that can be used to manage these circumstances and maintain tight control over your loop.

Break Example

```
for i in range(10):
 If i > 3:
  break
 print(i)
```

© Copyright 2026 MIVA Open University All Rights Reserved

Continue Example

```
for i in range(10):
 if i == 3:
  continue
 print(i)
```

Conclusion

In Python, decision-making is facilitated by the IF-Else and Nested IF-Else statements, which determine whether a block of statements should be executed based on a condition, termed "conditional statements". Decision and loop statements provide program control by addressing conditions and repetitive execution. The FOR and WHILE loops handle repetition in Python, while RANGE, BREAK, and CONTINUE statements offer control mechanisms within loops.

Summary

1. Decision-making in programming involves specifying actions based on conditions encountered during program execution. Python offers constructs like IF-Else and Nested IF statements for decision-making, facilitating programme control based on evaluated expressions.
2. Python's IF-ELSE statement allows for the execution of code based on specific conditions, while the nested IF statement permits additional condition checking based on preceding true conditions, enabling hierarchical decision-making.

© Copyright 2026 MIVA Open University All Rights Reserved

3. Loops facilitate repetitive execution of code blocks. The FOR loop iterates over a sequence, and the WHILE loop executes as long as a condition holds true. The Range Function generates a series of numbers within a specified range, while the Break Statement halts loop execution prematurely when a condition is met. The continue statement skips the current iteration of a loop based on a specified condition.

Self-Assessment Questions and Answers (SAQs and SAAs)

SAQ 3.1: Describe the syntax and usage of the FOR loop in Python.

Answer: The FOR loop in Python iterates over a sequence. The syntax is:

```
for item in sequence:
  statement(s)
```

Example:

```
for x in range(3, 8):
  print(x)
```

SAQ 3.2: What is the role of the range() function in Python, and how can it be used with loops?

Answer: The range() function generates a sequence of numbers that can be used in loops. Syntax:

```
range(start, stop, step)
```

Example:

```
for i in range(2, 10, 2):
  print(i)
```

© Copyright 2026 MIVA Open University All Rights Reserved

SAQ 3.3: How does the WHILE loop differ from the FOR loop, and when would you use each?

Answer: The WHILE loop runs as long as a condition is true, while the FOR loop iterates over a sequence or range. Use the WHILE loop when you don't know how many iterations are needed in advance and the FOR loop when you have a known sequence or range.

Multiple-Choice Questions (MCQs)

1. **What is the output of the following code?**

```
python
x = 10
if x > 5:
  print('Greater')
else:
  print('Smaller')
```

A. Greater

B. Smaller

C. Error

D. None

Answer: A

Explanation: The condition x > 5 is true, so 'Greater' is printed.

© Copyright 2026 MIVA Open University All Rights Reserved

2. **In Python, which loop is best suited for iterating over a fixed sequence of numbers?**

   A. WHILE loop

   B. FOR loop

   C. IF statement

   D. None

   Answer: B

   Explanation: The FOR loop is designed for iterating over sequences or ranges of numbers.

3. **Which Python keyword is used to exit a loop prematurely?**

   A. continue

   B. pass

   C. break

   D. exit

   Answer: C

   Explanation: The 'break' keyword is used to exit a loop prematurely when a condition is met.

4. **What is the purpose of the continue statement in Python loops?**

   A. It exits the loop.

   B. It skips the current iteration of the loop.

   C. It stops the program.

   D. It checks if the loop is complete.

   Answer: B

© Copyright 2026 MIVA Open University All Rights Reserved

Explanation: The continue statement causes the loop to skip the current iteration and proceed to the next.

5. **How does the range() function behave if no arguments are provided?**

   A. Starts at 1 and ends at 10.

   B. Starts at 0 and ends at 10.

   C. Starts at 0 and ends at infinity.

   D. Returns an empty sequence.

   Answer: D

   Explanation: The range() function requires at least one argument. Calling range() with no arguments would result in a TypeError. If one argument is provided (e.g., range(10)), it starts at 0 and ends at 9.

Tutor-Marked Assessments (TMAs)

1. Compare the FOR loop and the WHILE loop in Python. Discuss scenarios where each loop would be more appropriate to use.
2. Write a Python program that uses both the FOR loop and the IF-Else statement to print whether each number in a given range is even or odd.
3. Discuss the role of the break and continue statements in controlling loop execution. Provide examples of when each would be used.
4. Evaluate the efficiency and readability of nested loops and decision structures in Python. Discuss how code maintainability can be impacted by overly complex nesting.

© Copyright 2026 MIVA Open University All Rights Reserved

Study Session Four

Functions and Arrays

© Copyright 2026 MIVA Open University All Rights Reserved

Introduction

A function in Python is a collection of connected statements that carry out a certain activity. Functions assist in segmenting our program into manageable, modular portions in a more organised and controlled way as it gets bigger and bigger. It also makes the code reusable and prevents repetition. There are two types of functions: built-in functions (built in Python) and user-defined functions (defined by users).

Beyond functions, another essential concept in programming is the ability to work with collections of data. Often, we need to store and manipulate multiple related values under a single name—for example, a list of student scores, temperatures recorded over a week, or coordinates on a grid. This is where arrays come in. Arrays allow us to organise data efficiently, making it easier to process large amounts of information using loops and other control structures. In Python, we often use lists to achieve array-like functionality, and we can work with one-dimensional arrays (like a simple list) and two-dimensional arrays (like a table or matrix).

In this session, we will first explore how to define and use functions in Python, understanding parameters, arguments, return values, and scope. We will then dive into arrays, learning how to create, access, and manipulate data in both one and two dimensions. By the end of this session, you will be able to write modular programs using functions and handle collections of data efficiently using arrays.

© Copyright 2026 MIVA Open University All Rights Reserved

Learning Outcomes for Study Session 4

When you have studied this session, you should be able to:

4.1. Write a function in Python that can be implemented in the main program.

4.2. Explain what an array is and the types of arrays.

4.3. Execute codes using one-dimensional and two-dimensional arrays.

4.4. Differentiate between built-in functions and user-defined functions with examples.

4.5. Demonstrate how to pass data to functions using parameters and return results.

© Copyright 2026 MIVA Open University All Rights Reserved

4.1. Functions

A function in Python is a collection of connected statements that carry out a certain activity. Functions assist in segmenting our program into manageable, modular portions in a more organised and controlled way as it gets bigger and bigger. It also makes the code reusable and prevents repetition. There are two types of functions: built-in functions (built in Python) and user-defined functions (defined by users).

4.1.1. Declaration of Function

Syntax of Function

```
def function_name(parameters):
    statement(s)
```

The declared function contains the following:

a) It begins with the keyword 'def' in the function header.
b) A name for the function that serves as a unique identifier. The conventions for creating Python identifiers apply to naming functions as well.
c) The variables (arguments) that we use to supply values to a function. They are optional.
d) The function header ends with a colon (:).
e) Documentation string (docstring) that is optionally provided to explain the function's purpose.

© Copyright 2026 MIVA Open University All Rights Reserved

f) The body of the function is composed of one or more legitimate Python statements. The indentation level for each statement must be the same (usually 4 spaces).

g) A return statement that is optional that allows the function to return a value.

Example of Declaring a Function

```
def my_function():
 print("Hello from a function")
```

Example of Calling a Function

```
def my_function():
 print("Hello from a function")

my_function()
```

4.1.2. Scope of Function

How variables and names are looked up in your code is governed by the idea of scope. It establishes a variable's visibility within the code. The location in your code where you create a name or variable determines its scope.

A function's defined parameters and variables are not accessible from outside the function. They, therefore, have a localised scope. There are two scopes:

Global scope: All of your programs can access the names that you define in this scope.

© Copyright 2026 MIVA Open University All Rights Reserved

Local scope: Only the code that is included in the scope will have access to or be able to see the names you define.

Example of Local Scope:

```
def myfunc():
 x = 300
# Print x within the function
 print(x)
myfunc(),
```

```
def greet():

    # local variable
    message = 'Hello'

    print('Local', message)

greet()

# try to access message variable
# outside greet() function
print(message)
```

Run Code »

Output

```
Local Hello
NameError: name 'message' is not defined
```

Figure 4.1: Example Code of Local Scope

Example of Global Scope:

```
x = 300
```

© Copyright 2026 MIVA Open University All Rights Reserved

```
def myfunc():
 print(x)
#print x within the function
myfunc()


#print x outside the function


print(x)
```

```
# declare global variable
message = 'Hello'

def greet():
    # declare local variable
    print('Local', message)

greet()
print('Global', message)
```

Run Code »

Output

```
Local Hello
Global Hello
```

Figure 4.2: Example Code of Global Scope

© Copyright 2026 MIVA Open University All Rights Reserved

In-text Questions and Answers (ITQs and ITAs)

ITQ 4.1: What are the two types of functions in Python, and how do they differ?

Answer: The two types of functions in Python are built-in functions (which are provided by Python) and user-defined functions (which are created by the programmer). Built-in functions like print() or input() are predefined, while user-defined functions are written using the def keyword to perform specific tasks.

ITQ 4.2: What is the scope of a variable in Python, and how does it affect its accessibility?

Answer: The scope of a variable refers to where it can be accessed in the program. A variable defined within a function has local scope and is not accessible outside of it. A variable defined outside any function has global scope and can be accessed anywhere in the program.

4.2. Arrays

An array is a container that can house a certain number of elements, all of which must belong to the same type. Arrays are used by the majority of data structures to carry out their algorithms. The keywords for comprehending the idea of an array are “element” and “index”. The term "element" refers to each item kept in an array. An array's elements are identified by a number “index” that is assigned to each place of the element.

When you need to use numerous variables of the same type, Python arrays are employed. Additionally, it can be used to store a set of data. When processing

© Copyright 2026 MIVA Open University All Rights Reserved

data dynamically, arrays are very helpful. Python arrays consume less memory than lists, making them much faster.

4.2.1. Declaration of Arrays

Arrays are declared following this syntax:

```
const array_name = [item1, item2, ...];
```

The keyword const was introduced in Python in 2015. Arrays declared with the keyword can not be reassigned. The elements in the array are separated by a comma and grouped by square brackets.

The figure below shows how the element in the array can be accessed. Note that the numbering of the elements begins with 0.

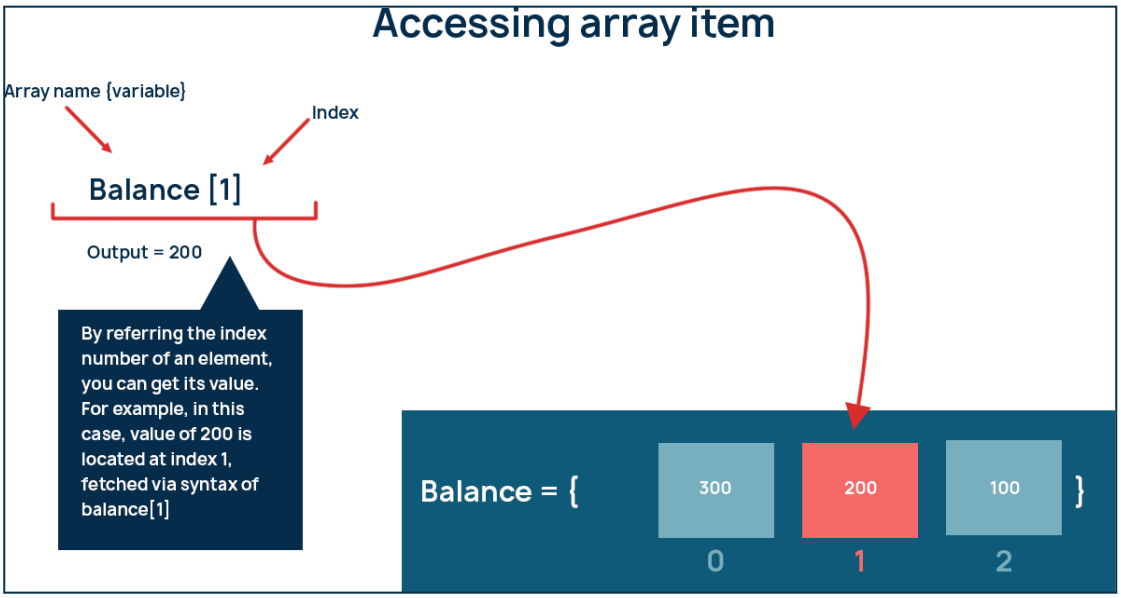


Figure 4.3: Array

© Copyright 2026 MIVA Open University All Rights Reserved

4.2.2. Types of Arrays

There are one-dimensional and two-dimensional arrays in Python. The following are the characteristics of a one-dimensional array:

a) It is a collection of variables with related data types.
b) All of the elements can be retrieved by their index, and it enables random access.
c) The array's size is fixed.

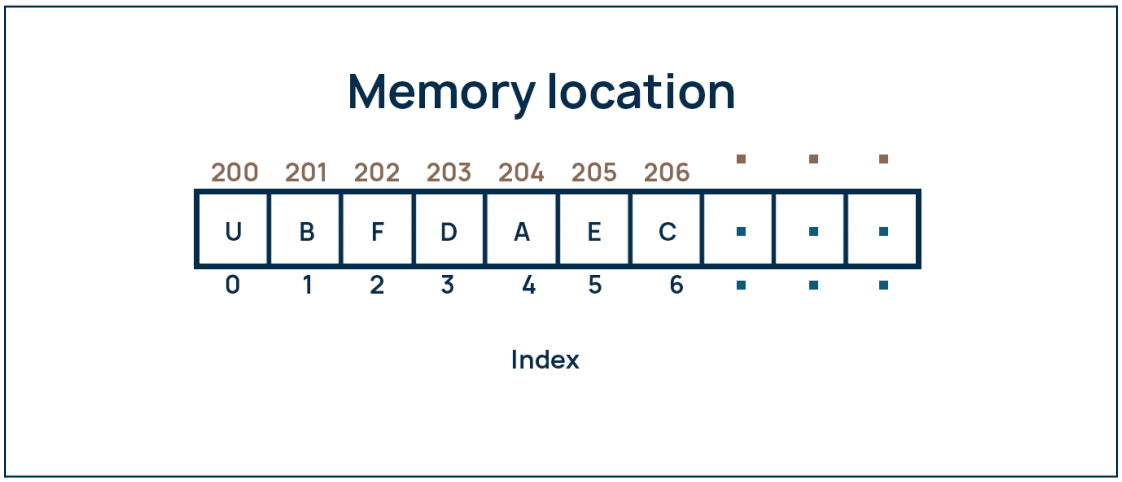


Figure 4.2 One-directional array

Example code of a one-dimensional array

```
x = [1, 2, 3, 4, 5, 6, 7]
for element in x:
  print(element)
```

The two-dimensional arrays have the following characteristics:

a) It is a list of lists that share the same data type as the variable.

© Copyright 2026 MIVA Open University All Rights Reserved

b) All of the elements can be accessed with the aid of their index, and it also permits random access.

c) It may alternatively be thought of as a group of 1D arrays. The Matrix is another name for it

d) Its dimension can be raised from two to three, then four, and so on.

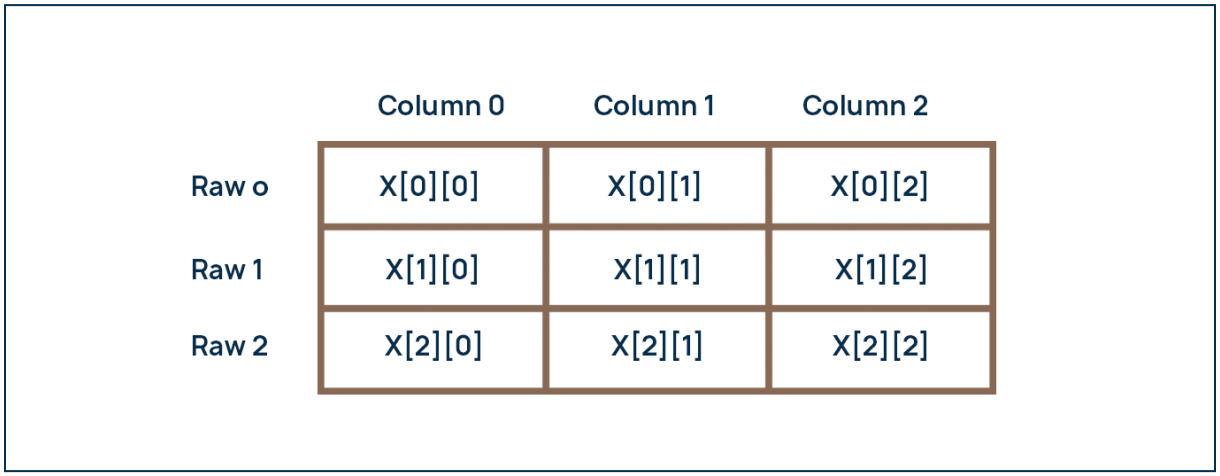


Figure 4.3 Two-directional array

Example Code of 2-dimensional Arrays

```
# Create a 2D array with 4 rows and 5 columns
array=[[23,45,43,23,45],[45,67,54,32,45],[89,90,87,65,44],[23,45,67,32,10]]

#use for loop to iterate the array
for rows in array:
 for columns in rows:
  print(columns, end=" ")
  print()
```

© Copyright 2026 MIVA Open University All Rights Reserved

Array Methods

The table below shows array inbuilt methods that can be used for manipulating a list.

Table 4.1: Array Inbuilt Methods For Manipulating A List

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable) to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the first item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

© Copyright 2026 MIVA Open University All Rights Reserved

Conclusion

In Python, functions serve the dual purpose of facilitating code reuse and organisation, thereby mitigating the need for writing lengthy programs with excessive lines of code (LOC). Additionally, arrays play a vital role in Python programming by enabling the arrangement of similar items, with support for both one-dimensional and multiple-dimensional arrays. Furthermore, Python offers a range of built-in functions specifically designed for array manipulation, enhancing the efficiency and flexibility of array operations within programs.

Summary

1. Functions in Python streamline code organisation and reuse, with built-in and user-defined functions providing modular solutions to repetitive tasks, enhancing programme structure and readability.
2. Arrays in Python serve as containers for storing elements of the same data type, enabling efficient data organisation and manipulation. They include one-dimensional and two-dimensional arrays, accessible through indexing and supporting a variety of built-in methods for manipulation.
3. Python provides a robust framework for declaring functions and arrays, with functions defined using the 'def' keyword and arrays declared within square brackets. The scope of variables within functions is determined by their local or global nature, and arrays offer efficient memory usage and manipulation capabilities through built-in methods.

© Copyright 2026 MIVA Open University All Rights Reserved

Self-Assessment Questions and Answers (SAQs and SAAs)

SAQ 4.1: What is the difference between local scope and global scope in Python? Provide examples.

Answer: In Python, variables in the global scope can be accessed anywhere in the program, while variables in the local scope are accessible only within the function where they are defined.

Example of local scope:

```
def myfunc():
    x = 300
    print(x)  # x is accessible within the function
myfunc()
```

Example of global scope:

```
x = 300
def myfunc():
    print(x)  # x is accessible globally
myfunc()
print(x)  # x can also be accessed outside the function
```

SAQ 4.2: What is the purpose of functions in Python, and how do they improve code structure?

Answer: Functions improve code structure by modularising code, making it reusable and reducing repetition. They allow for logical segmentation of a programme, making it more organised and easier to maintain and test.

© Copyright 2026 MIVA Open University All Rights Reserved

SAQ 4.3: Explain the concept of arrays in Python and their memory efficiency compared to lists.

Answer: Arrays are collections of elements of the same type, and they use less memory compared to lists because they are more space-efficient, especially when handling large amounts of data. Arrays also allow for faster access and manipulation of data.

Multiple-Choice Questions and Answers (MCQs and MCAs)

1. **What is the purpose of the 'def' keyword in Python?**

   A. To define a class

   B. To declare a function

   C. To define a variable

   D. To create an array

   Answer: B

   Explanation: The 'def' keyword is used in Python to define a function.

2. **How do you declare a one-dimensional array in Python?**

   A. array_name = {1, 2, 3, 4}

   B. array_name = [1, 2, 3, 4]

   C. array_name = (1, 2, 3, 4)

   D. array_name = {1:2, 3:4}

   Answer: B

   Explanation: Arrays in Python are declared using square brackets, e.g., [1, 2, 3, 4].

© Copyright 2026 MIVA Open University All Rights Reserved

3. **What is the default scope of variables in Python if not specified?**

   A. Global scope

   B. Local scope

   C. Both global and local scope

   D. No scope

   Answer: A

   Explanation: By default, variables defined outside functions have a global scope.

4. **Which of the following is NOT an array method in Python?**

   A. append()

   B. clear()

   C. copy()

   D. extend()

   Answer: D

   Explanation: extend() is a list method, not an array method in Python. Arrays in Python have different methods.

5. **What will be the output of the following code?**

   Python

   ```
   x = [10, 20, 30]
   x[1] = 25
   print(x)
   ```

   A. [10, 20, 30]

   B. [10, 25, 30]

© Copyright 2026 MIVA Open University All Rights Reserved

C. [25, 20, 30]

D. [10, 20, 25]

Answer: B

Explanation: The second element of the list (index 1) is updated to 25, so the list becomes [10, 25, 30].

Tutor-Marked Assessments (TMAs)

1. Explain the concept of scope in Python and how it affects variable accessibility within functions. Provide examples of both local and global scope.
2. Write a Python program that defines a function to calculate the area of a rectangle. The function should take two parameters (length and width) and return the calculated area.
3. Compare and contrast one-dimensional and two-dimensional arrays in Python. Provide examples where each type of array would be used.
4. Write a Python program that uses a two-dimensional array to represent a matrix. Implement a function that adds two matrices and prints the resulting matrix.

© Copyright 2026 MIVA Open University All Rights Reserved

Study Session Five

File Handling

© Copyright 2026 MIVA Open University All Rights Reserved

Introduction

To store linked data, files are named places on a disc. They serve as a permanent data storage device in non-volatile memory (e.g., a hard disc). Since files can be used for future use of the data by being permanently stored, we use them instead of volatile Random Access Memory (RAM), which loses its contents when the computer is turned off. A file must first be opened before it can be read from or written to. When we are finished, it must be closed to release the resources used by the file.

In any real-world programming scenario, data must persist beyond the execution of a single program. Whether you are building a simple contact book, a data logging application for sensors, or a web application that stores user preferences, the ability to read from and write to files is essential. Python provides built-in functions and methods that make file handling straightforward and intuitive. However, working with files also introduces risks: the file you want to read might not exist, you might not have permission to write to a location, or the disc might be full. These runtime errors, if unhandled, will cause your program to crash. This is why exception handling is a critical companion to file operations. By learning to anticipate and manage errors using try and except blocks, you will be able to write programs that are not only functional but also robust and user-friendly. This session will guide you through both opening, reading, writing, and closing files, as well as implementing comprehensive exception handling strategies to deal with the unexpected.

© Copyright 2026 MIVA Open University All Rights Reserved

Learning Outcomes for Study Session 5

When you have studied this session, you should be able to:

5.1. Explain Python file operations.

5.2. Apply various file-handling techniques to open, read, write, and close files in Python programs.

5.3. Implement exception-handling mechanisms using `try` and `except` statements to detect and correct runtime errors.

5.4. Develop programmes that handle multiple exceptions and create user-defined exceptions effectively.

© Copyright 2026 MIVA Open University All Rights Reserved

5.1. How To Read, Write And Append Files

5.1.1. Opening a File

We must first open the file in question before we can read from it or write to it. To do this, we should use the built-in Python function open(), but we must first give the mode, which denotes the goal of the opening file.

Syntax

```
f = open("demofile.txt")
```

The following modes are used for opening a file:

r: read an existing file by opening it.

w: initiates a write operation on an open file. If the file already has some data in it, it will be overwritten; if it doesn't exist, it will be created.

a: start an add operation on an existing file. Existing data won't be replaced by it.

r+: to both read from and write to the file. It will replace any prior data in the file.

w+: to read and write data It will replace current data.

a+: to add to and read from the file with data. Existing data won't be replaced by it.

Example

```
f = open("test.txt")     # similar to 'rt' or 'r'
f = open("img.bmp,r+b')  # read and write in binary mode
f = open("test.txt,w')  # write in text mode


Open a file in a different location:
f = open ("D:\\Nigeria\welcome.txt", "r")
print(f.read())
```

© Copyright 2026 MIVA Open University All Rights Reserved

5.1.2. Writing a File

Write mode establishes a file for writing content and starts at the beginning. If the file already exists, writing truncates (clears) all data in it.

Syntax

```
fileObject.write(string)
```

Example

```
# To open a file for writing, use:
f = open("<file name>", "w")

This is similar to the above code:
f = open("<file name>", "wt")

To write in binary mode, open the file with:
f = open("<file name>", "wb")
```

5.1.3. Reading a File

Python provides a variety of ways to read a file's contents after importing it into an object. Print the outcome after using the file object's read() method.

Syntax

```
fileObject.read([count])
```

Example

```
 n = open("filename.txt")
print(fn.read(), end="")

# Read Parts of the File
```

© Copyright 2026 MIVA Open University All Rights Reserved

```
fn = open("filename.txt")
print(fn.read(5))
```

When interacting with file objects, Python provides a number of additional functions. The list of all available processes and their functions may be found in the table below:

Table 5.1: File Object Processes and Functions

Method	Description
close()	Flushes and closes the file object.
detach()	Separates the buffer from the text stream and returns the buffer.
fileno()	Returns the file's descriptor if available.
flush()	Flushes the write buffer. Not available for read-only objects.
isatty()	Checks if a file stream is interactive.
read(<int>)	Read <int> number of characters at most.
readable()	Checks if an object is readable.
readline(<int>)	Reads from the object until a newline or the end of the file.
readlines(<int>)	Returns a list of lines from the file object, where <int> is the approximate character number.

© Copyright 2026 MIVA Open University All Rights Reserved

seek(<offset>, <position>)	Changes the pointer position to **<offset>** relative to the **<position>**.
seekable()	Checks if the file object supports random access.
tell()	Prints the current stream position.
truncate(<byte>)	Resizes the file stream to **<bytes>** (or the current position if unstated) and returns the size.
write(<string>)	Writes **<string>** to the file object and returns the written number of characters.
writable()	Checks whether the file object allows writing.
writelines(<list>)	Writes a **<list>** of lines to the stream without a line separator.

In-text Questions and Answers (ITQs and ITAs)

ITQ 5.1: What are the different file modes used in Python for opening a file?

Answer: The different file modes in Python are:

- **r:** Read an existing file.
- **w:** Write to a file (overwrites existing data).
- **a:** Append to an existing file.
- **r+:** Read and write to a file (overwrites data).
- **w+:** Read and write to a file (overwrites existing data).
- **a+:** Append and read from a file.

© Copyright 2026 MIVA Open University All Rights Reserved

ITQ 5.2: How can a file be opened in a specific location in Python?

Answer: A file can be opened in a specific location by providing the full file path in the open() function, for example:

```
f = open ("D:\\Nigeria\\welcome.txt", "r")
```

5.2. Exception Handling

An event that occurs during the execution of a program and obstructs the regular flow of the program's instructions is an exception. A Python script typically raises an exception when it comes to a scenario that it cannot handle. A Python object that describes an error is called an exception. A try statement in Python can be used to manage exceptions. The try statement contains the crucial operation that can cause an exception. The except statement contains the code that manages exceptions. Thus, once we have identified the exception, we may decide what actions to take.

5.2.1. Catching Exceptions

If your program contains any questionable code that could cause an exception, you can protect it by enclosing it in a try block. Include an except statement after the try block, then a block of code that solves the issue as simply as feasible.

© Copyright 2026 MIVA Open University All Rights Reserved

Syntax

```
try...except...else blocks—
try:
  code block;
  ....................
except e1:
  If there is e1, then execute this block.
except e2:
  If there is e2, then execute this block.
  ..................
else:
  If there is no exception, then execute this block.
```

Example

```
try:
  nfile = open("nfile", "w")
  nfile.write("Testing for exception handling!!")
except IO_Error:
  print "Can't find file"
else:
  print " file content is written successfully"
  nfile.close()
```

© Copyright 2026 MIVA Open University All Rights Reserved

5.2.2. Except Clause with No Exceptions

The following is the definition of the 'no exceptions' statement for the 'except' statement. The syntax is as follows:

```
try:
  put your statement block here.
  ....................
except:
Execute this block if there is an exception.
  ....................
else:
Execute this block if there is no exception.
```

5.2.3. Multiple Exceptions handling

Multiple exceptions can be handled as follows:

```
try:
  put your statement block here.
  ....................
except(Exception1[, Exception2[,...ExceptionN]]):
Execute this block if there is an exception from the given exception list
  ....................
else:
Execute this block if there is no exception from the given exception list
```

© Copyright 2026 MIVA Open University All Rights Reserved

```
try:
    x = int(input("Enter a number: "))
    result = 10 / x
except (ValueError, ZeroDivisionError):
    print("Invalid input or division by zero")
else:
    print("Result:", result)
```

Figure 5.1: Multiple exceptions handling code example

5.2.4. User-Defined Exceptions

You may also make your exceptions in Python by deriving classes from the default built-in exceptions. Here is a case study with a runtime error. In this case, a subclass of RuntimeError is produced. This is helpful when you need to show more detailed information after catching an exception. The user-defined exception is raised and handled in the try-except block. An instance of the class NetworkError is created using the variable e.

```
class NetworkError (RuntimeError)
 def ___init___(self, arg)
  self.args = arg
```

Conclusion

Python simplifies file management with built-in methods for creating, opening, and closing files, making tasks like reading, writing, and appending data

© Copyright 2026 MIVA Open University All Rights Reserved

straightforward. Additionally, Python's exception handling feature helps manage unexpected issues in code, ensuring error-free programming and facilitating effective debugging by promptly identifying and addressing errors as they occur.

Summary

1. Python simplifies file management through built-in methods for file operations like reading, writing, and appending. Understanding file modes ('r', 'w', 'a', 'r+', 'w+', 'a+') and employing them correctly is crucial for efficient file handling.
2. Opening files using the open() function and writing data with the write() method are essential for file manipulation. Reading files with methods like read() and readline() helps retrieve data efficiently.
3. Exception handling in Python enables the management of errors that may arise during program execution. Employing try, except, else, and finally blocks allows programmers to gracefully handle exceptions and maintain program flow.

Self-Assessment Questions and Answers (SAQs and SAAs)

SAQ 5.1: Explain the difference between the write() and writelines() methods in Python file handling.

Answer: The write() method writes a string to a file, while the writelines() method writes a list of strings to a file. writelines() does not add line breaks automatically, so you need to include them manually.

Example:

© Copyright 2026 MIVA Open University All Rights Reserved

```
f.write("Hello")
f.writelines(["Hello\n", "World\n"])
```

SAQ 5.2: Describe the process of handling exceptions in Python and explain the role of the else block.

Answer: In Python, exceptions are handled using the try-except block. The try block contains code that may raise an exception, while the except block handles the exception. The else block executes if no exceptions are raised in the try block.

Example:

```
try:
  # Code that might raise an exception
except ExceptionType:
  # Code to handle the exception
else:
  # Code to run if no exception occurred
```

SAQ 5.3: What is the purpose of the close() method when working with files in Python?

Answer: The close() method is used to close a file after performing file operations. Closing a file releases system resources and ensures that all changes are saved properly.

Example:

```
f = open("test.txt", "w")
```

© Copyright 2026 MIVA Open University All Rights Reserved

```
f.write("Hello")
f.close()
```

Multiple-Choice Questions and Answers (MCQs and MCAs)

1. **Which file mode is used to read and write to a file without truncating the file in Python?**

   A. r

   B. w+

   C. a+

   D. r+

   Answer: D

   Explanation: The r+ mode allows both reading and writing to the file without truncating the file.

2. **What is the default behaviour of the open() function if no mode is specified?**

   A. Open the file for reading

   B. Open the file for writing

   C. Open the file for appending

   D. Raises an error

   Answer: A

   Explanation: If no mode is specified, the open() function defaults to reading the file.

3. **How can you prevent a program from crashing when an exception occurs in Python?**

© Copyright 2026 MIVA Open University All Rights Reserved

A. By using the try block

B. By using the finally block

C. By using the except block

D. By using the else block

Answer: C

Explanation: The except block handles the exception and prevents the program from crashing.

4. **Which of the following methods is used to read the entire content of a file in Python?**

A. readline()

B. read()

C. writelines()

D. seek()

Answer: B

Explanation: The read() method is used to read the entire content of a file.

5. **How do you handle multiple exceptions in Python?**

A. By using multiple try blocks

B. By using a single except block for each exception

C. By using a single except block and specifying multiple exceptions,

D. By using the finally block

Answer: C

Explanation: Multiple exceptions can be handled using a single except block by specifying multiple exception types.

© Copyright 2026 MIVA Open University All Rights Reserved

Tutor-Marked Assessments (TMAs)

1. Write a Python program that opens a file, reads its content, and prints it to the console. Include exception handling to manage errors such as the file not being found.
2. Discuss the different file modes used in Python when opening a file. Provide examples of when each mode should be used.
3. Write a Python function that accepts user input, writes the input to a file, and then reads the file to display the entered data.
4. Create a Python program that handles user-defined exceptions. Implement a custom exception that raises an error when a user enters an invalid input.

© Copyright 2026 MIVA Open University All Rights Reserved

References/Further Readings

GeeksforGeeks. (2022). *Difference between a one-dimensional and a two-dimensional array.* https://www.geeksforgeeks.org/difference-between-one-dimensional-and-two-dimensional-array/

Pankaj. (2022, August 3). *Python data types.* DigitalOcean. https://www.digitalocean.com/community/tutorials/python-data-types

Phoenixnap. (2022). *File handling in Python: Create, open, append, read, write.* https://phoenixnap.com/kb/file-handling-in-python

Programiz. (2022). *Files in Python.* https://www.programiz.com/python-programming/file-operation

Programiz. (2022). *Functions in Python.* https://www.programiz.com/python-programming/function

Programiz. (2022). *Python examples.* https://www.programiz.com/python-programming/examples

Python Software Foundation. (2021). *The Python documentation.* https://www.python.org/doc/essays/blurb/

Simplilearn. (2022, October 26). *Comments in Python: Why are they important and how do you use them?* https://www.simplilearn.com/tutorials/python-tutorial/comments-in-python

© Copyright 2026 MIVA Open University All Rights Reserved

Tutorialspoint. (2022). *Python decision-making.* https://www.tutorialspoint.com/python/python_decision_making.htm

Tutorialspoint. (2022). *Python exceptions.* https://www.tutorialspoint.com/python/python_exceptions.htm

Tutorialspoint. (2022). *Python for loop.* https://www.tutorialspoint.com/python/python_for_loop.htm

W3Schools. (2022). *Python arrays.* https://www.w3schools.com/python/python_arrays.asp

W3Schools. (2022). *Python continue statement.* https://www.w3schools.com/python/ref_keyword_continue.asp

W3Schools. (2022). *Python functions.* https://www.w3schools.com/python/python_functions.asp

W3Schools. (2022). *Python tutorial: Input function.* https://www.w3schools.com/python/ref_func_input.asp

© Copyright 2026 MIVA Open University All Rights Reserved

Glossary

Arrays: A data structure that stores a collection of items at contiguous memory locations, allowing easy access and manipulation of elements.

Comments: Text was added to the code for explanatory purposes and is ignored by the compiler or interpreter.

Data types: Classification of data items based on the kind of value they represent, such as integers, floats, strings, etc.

Decision Statement: Programming constructs that allow branching based on conditions, such as if-else statements.

Exception Handling: A mechanism to handle errors that occur during program execution gracefully, preventing abrupt termination.

File Handling: Operations involving reading from, writing to, and manipulating files stored on a computer's filesystem.

Functions: Self-contained blocks of code that perform a specific task, reusable throughout the program.

High-Level Language: Programming languages designed to be easy for humans to understand and write, abstracting away hardware-specific details.

Installation of Python: The process of setting up the Python programming language on a computer system.

© Copyright 2026 MIVA Open University All Rights Reserved

Input and Output Statements: Commands are used to take input from users and produce output in the form of text or data.

Low-Level Language (Assembly Language): Programming languages are closely related to machine code, representing instructions in a more human-readable format.

Machine Language: Binary code is understood directly by computer hardware, representing instructions and data.

Nested IF Statement: Using one if or else if statement inside another if or else if statement.

Operators: Symbols used to perform operations on operands, such as arithmetic, comparison, and logical operations.

Python: A high-level, interpreted programming language known for its simplicity and readability.

Range, Break, and Continue: Constructs used in loops to control their execution flow.

Scope of Function: The area in a program where a function can be accessed and used.

Types of Arrays: Different classifications of arrays based on their dimensions and data types they hold.

Types of Programming Language: Classification based on the level of abstraction and closeness to machine code.

© Copyright 2026 MIVA Open University All Rights Reserved

User-Defined Exceptions: Custom exceptions defined by the programmer to handle specific error conditions.

WHILE Loop: A loop construct that repeatedly executes a block of code as long as a specified condition is true.

© Copyright 2026 MIVA Open University All Rights Reserved